
COMPSCI 340

Tutorial 1

xv6 – key facts

- Modern reimplementations of Unix v6
- Designed for teaching Operating Systems – minimal & readable
- Runs on RISC-V architecture (via QEMU)
- Supports: processes, virtual memory, files, pipes, signals
- Originally written in C – roughly 6,000 lines of code
- Starting this semester, we will also be using a Rust port of xv6

C Language

- Developed alongside Unix at Bell Labs by Dennis Ritchie
- Low level and very simple language
 - Pointers allow direct access to memory
 - C code maps closely to generated assembly code
- Highly portable
 - Same code can be compiled for different platforms
 - C compilers exist for almost every architecture
- C is widely used in OS development
 - Kernels
 - Drivers
 - Networking

Memory Safety

In the C language memory needs to be explicitly allocated, managed, and freed by the programmer. In C code arrays can be accessed out of bounds if the programmer is not careful. This can lead to memory leaks and security vulnerabilities.

Garbage collected languages such as Python and Java solve this issue, but at a high runtime performance cost.

The Rust language aims to address these issues with minimal runtime overhead.

Rust Language

- Focuses on type safety, memory safety, encapsulation, and concurrency
- Type safety makes many logical errors and bugs into compilation errors
- The Rust compiler performs a lot of work to reduce the overhead of memory safety
 - Rust is generally as fast as C code at runtime
- Rust provides access to low level features with unsafe code blocks, which encapsulate code that could cause vulnerabilities

Rust Language – Borrow Checker

Rust's borrow checker is a system that ensures memory safety at compile time, without the runtime overhead of garbage collection. The borrow checker ensures that one single object owns a memory allocation, and prevents concurrent modification. This makes a number of common bugs impossible:

- Memory leaks
- Dangling pointers and use after free
- Double free
- Race conditions

Assignment 1 – Setting up a VM

- This assignment needs to be completed on a Unix System
- We will be marking the assignment on Ubuntu 24.04
- We recommend using Oracle VirtualBox to set up and Ubuntu 24.04 VM
- You can also use FlexIT
 - University service that allows access to VMs and other software
 - You won't have to set up the VM itself

Assignment 1 – Versions

You can choose to complete the assignment in either **C** or **Rust**.

- There is no difference in what you will be implementing
- You will need to complete the whole assignment in one language
- You can switch languages for different assignments
- It is up to you which language you choose
 - Neither language is easier than the other

Assignment 1 – Getting Started

1. Set up a Unix development environment
2. Install the required tools
3. Clone the appropriate git repository
4. Follow the instructions to compile and run xv6 via QEMU

Booting C Version

```
# Clone the repo
git clone https://github.com/UoA-CS340-SE370-OSDev/xv6-
c-student.git

# Enter directory
cd xv6-labs

# Build & run
make qemu

# Exit QEMU
Ctrl-a then x

# View running processes
Ctrl-p
```

Assignment 1 – Tips

- Get your development environment set up early
- Take a look at the existing programs
 - The entire OS is small enough to read through
- You can't use the standard library
 - You will need to use the functions provided by the xv6 kernel
 - With Rust you will need to add `#![no_std]` and `#![no_main]`
- If you're stuck – Ask on EdDiscussion